Textual Analysis

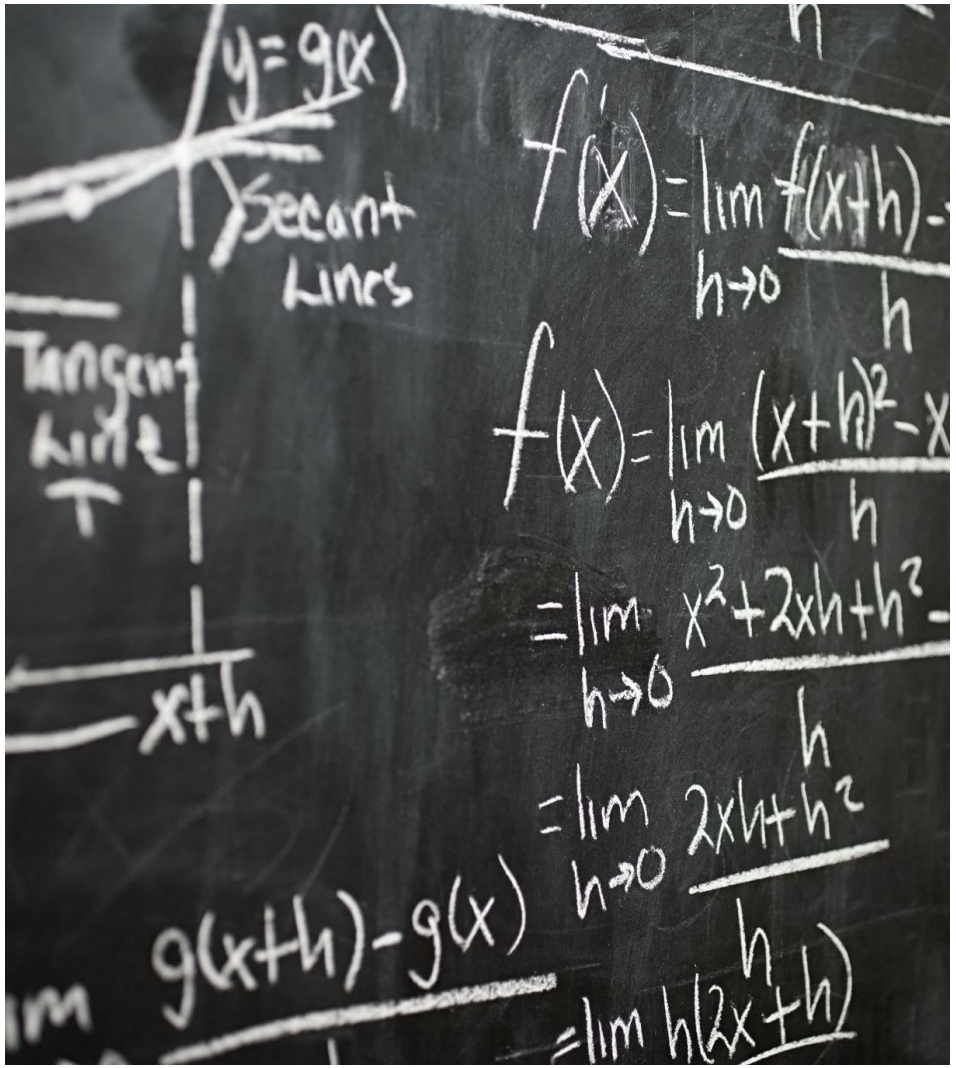
Kai Yang, kaiyang@ln.edu.hk

24 October 2025

- From Text to Data:

  We move from qualitative reading to quantitative counting to find patterns

  - Core Goal: To show how to transform raw text into a structured, quantitative format and perform foundational analysis.



Part 1: Introduction & The Big Picture

- Question:
- "How can we measure the sentiment of 10,000 product reviews without reading them all?"
- "Can we find the main themes in the Chief Executive (CE)’s policy address automatically?"

- We treat each document as a collection of features (words) that we can count and analyze statistically.

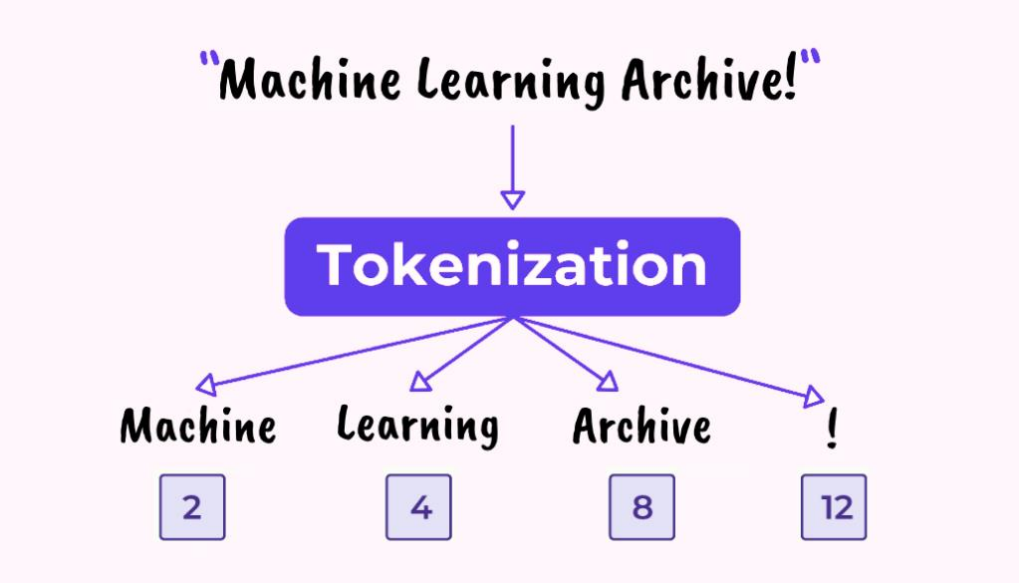
- The Bag-of-Words (BoW) model: a sentence and its representation as a table of word counts.

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Part 2: The Text Processing

- Cleaning and preparing raw ingredients before cooking:

- Step 1: Tokenization
- Splitting text into individual units (tokens), usually words



- *Tokenization is not always about single words, and the process is very different for English vs. Chinese.

- While splitting text into individual words (unigrams) is the most common approach, tokenization can happen at different levels:

- I. Word-Level Tokenization (Most Common)
- English: "I don't like pizza." → ["I", "do", "n't", "like", "pizza", "."] (Note how "don't" is split)
- This is what people typically think of.

- II. Subword Tokenization (Increasingly Important)
- Splits words into smaller meaningful units. Crucial for handling rare words and morphologically rich languages.
- Example: "unhappiness" → ["un", "happiness"] or ["un", "happy", "ness"]
- Used by modern models like BERT and GPT to handle vast vocabularies efficiently.

- III. Character-Level Tokenization
- Treats each character as a token.
- Example: "cat" → ["c", "a", "t"]
- Useful for languages without clear word boundaries or for robust spelling.

- IV. N-gram Tokenization (Multi-word Terms)
- Captures phrases by creating tokens of N consecutive words.
- Bigrams (2-grams): "I love New York" → ["I love", "love New", "New York"]
- Trigrams (3-grams): ["I love New", "love New York"]
- It helps capture multi-word concepts like "New York" or "social media"

- Chinese Tokenization: much more complicated…

- Word Boundaries: None. Text is a continuous stream of characters.
- English: "I | love | New | York" (spaces show boundaries)
- Chinese: "我愛紐約" (No spaces: 我愛紐約 = "I love New York")

- Tokenization in Chinese is called Word Segmentation (中文分词).
- The algorithm must infer where words start and end based on context and a dictionary

- Famous Ambiguity Example: "美國會根據法律行事"
- Possible Segmentations:
- "美國(USA) | 會(will) | 根據(according to) | 法律(law) | 行事(act)“: ("The USA will act according to the law.")
- "美(America) | 國會(Congress) | 根(root) | 據(according to) | 法律(law) | 行事(act)“: (A nonsensical reading: "America Congress root according to law act")
- The correct segmentation depends entirely on context, which requires sophisticated models.

- Step 2: Normalization & Cleaning

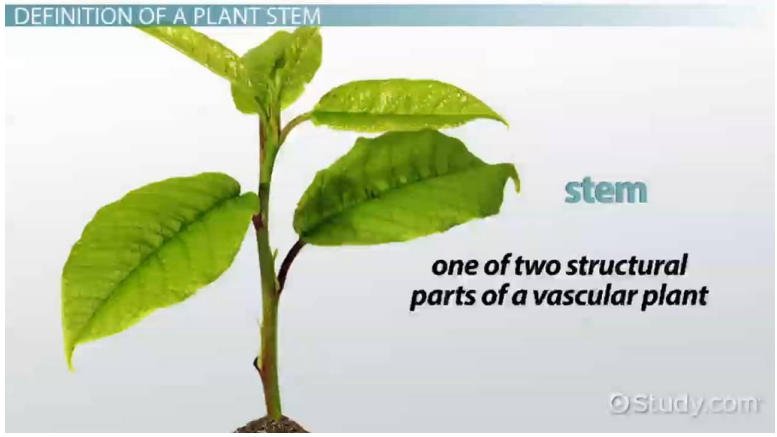
- Lowercasing: "The" and "the" are the same word

- Removing Punctuation/Numbers: (Context-dependent: "COVID-19" might be important!)

- Stopword Removal:
- What: Filtering out extremely common words ("the", "and", "is").
- Why: They add noise and little semantic meaning in a BoW model.

- Step 3: Lemmatization (or Stemming)

- Goal: Reducing words to their base form.



- Stemming (cruder): "running" -> "run", "argued" -> "argu"
- Lemmatization (smarter): "running" -> "run", "better" -> "good", "is" -> "be".

- Recommendation: Focus on Lemmatization as it's more accurate for social science.

Part 3: From Words to Numbers & Basic Analysis

- I. The Bag-of-Words Matrix:
- Visualize an example with 3 documents and a small vocabulary. Show it as a DataFrame where rows are documents and columns are words, with counts in the cells.

Document D1	*The child makes the dog happy* the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	*The dog makes the child happy* the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	**[1,1,1,1,2]**
D2	1	1	1	1	2	**[1,1,1,1,2]**

- II. Term Frequency (TF):

- This is the raw count in the BoW matrix.
- Insight: Simple frequency can reveal important words within a document.

$$TF(t, d) = \frac{\text{Number of times term t appears in document d}}{\text{Total number of terms in document d}}$$

- III. TF-IDF (Term Frequency-Inverse Document Frequency):

- The Problem with Raw TF: The word "the" might be the most frequent in every document, but it's not informative.

- The TF-IDF Solution: It balances:
- TF: How often a word appears in this document.
- IDF: How unique the word is across all documents.

- Intuitive Explanation: TF-IDF identifies words that are important to a specific document but not common across the entire collection.

- Example: In a corpus of sports articles, the word "goalkeeper" might have a high TF-IDF score in one article about soccer, but a low score for the common word "win"

- Term Frequency (TF):

$$TF(t, d) = \frac{\text{Number of times term t appears in document d}}{\text{Total number of terms in document d}}$$

- Inverse Document Frequency (IDF): Reduces the weight of common words across multiple documents while increasing the weight of rare words.

$$IDF(t, D) = \log \frac{\text{Total number of documents in corpus D}}{\text{Number of documents containing term t}}$$

- TF-IDF = TF*IDF

Part 4: A Powerful Application: Dictionary Methods

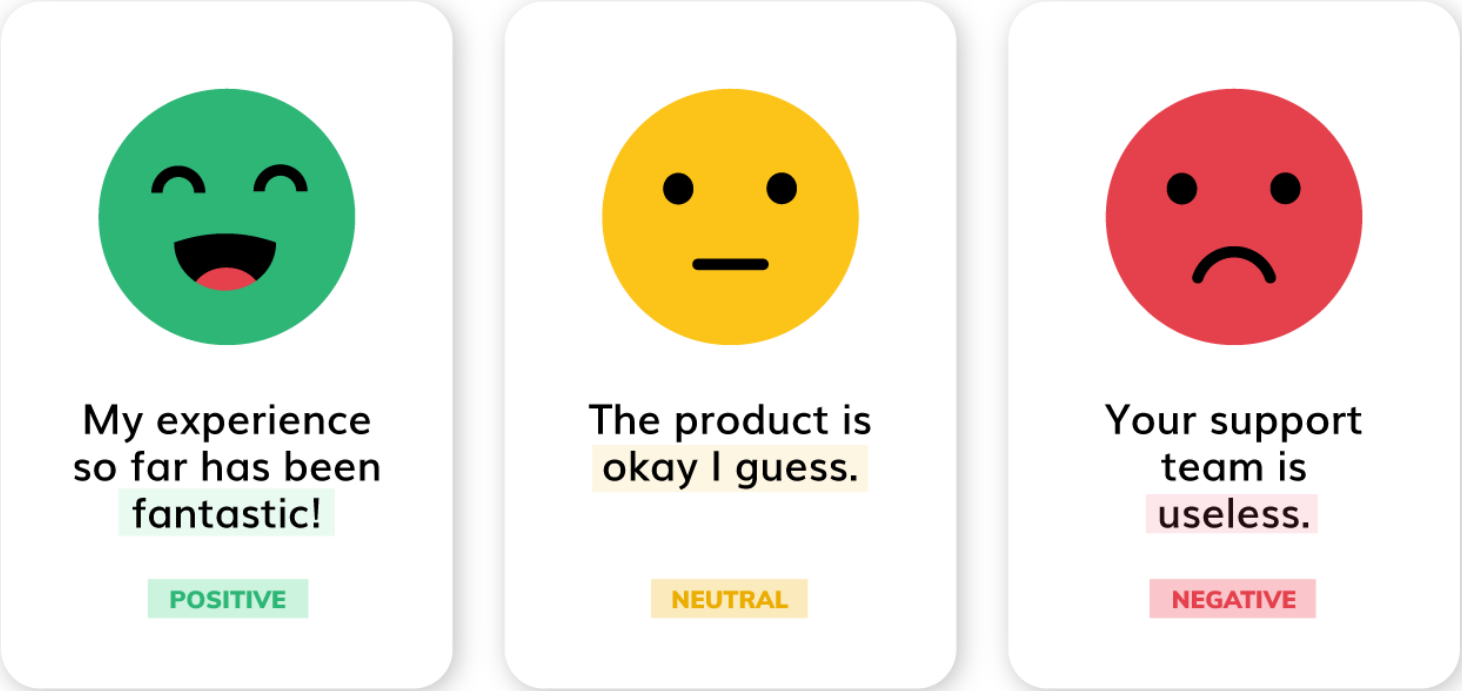
- Concept: Using a pre-made "dictionary" (word list) to measure a concept.

- Star Example: Sentiment Analysis

- How it works:
- You have a dictionary of positive words ("good, great, amazing") and negative words ("bad, terrible, awful").
- Process your text (tokenize, clean, lemmatize).
- Count the words that match each list.

- Score = (Positive Count - Negative Count) / Total Words.

Thank you!


My experience so far has been fantastic!
POSITIVE
The product is okay I guess.
NEUTRAL
Your support team is useless.
NEGATIVE